SF2565/FSF3565 Assignment 2

In this assignment, we will explore some basic techniques in digital image processing. We will build a flexible implementation that will apply various image transformation steps in a pipeline, using object-oriented programming.

In addition, you will get some experience integrating a C library in a C++ project, applying proper resource management techniques to avoid memory leaks and other problems.

Digital image processing

We begin by giving some background on digital image processing. For a more complete account, see, e.g., [1], on which much of this section, as well as the remaining image processing parts of the following text are based.

Images as signals

We will think of a grayscale image as a *signal* $f\colon \mathbb{R}^2 \to [0,1]$. A color image can then be thought of as a mapping $\mathbb{R}^2 \to [0,1]^3$, i.e., three grayscale images that correspond to the red, green and blue channels of the image, each represented as a dimension in the output vector.

To enable some more sophisticated techniques later[1], we will use $\mathbb{C}$ as the target space instead of $[0,1]$. This is no impediment since $[0,1] \subset \mathbb{C}$, so we can use f as stated above. However, going forward, we will think of f as a function $\mathbb{R}^2 \to \mathbb{C}$.

Now, in reality, an image has a finite number of pixels. We will view the pixel values as *samples* taken over the input space, $\mathbb{R}^2$.

To make mathematical sense of this, let's introduce the Dirac delta distribution, $\delta(\mathbf{x})$, which can be defined nonrigorously[2] as

$$\delta(\mathbf{x}) \coloneqq \begin{cases} \infty & \text{if } \mathbf{x} = 0, \\ 0 & \text{otherwise,} \end{cases}$$

together with the property

$$\int_{\mathbb{R}^2} \delta(\mathbf{x})\, d\mathbf{x} = 1.$$

[1] Namely, we want to be able to work in the frequency/Fourier domain, where our signals will be complex-valued.

[2] A more rigorous definition is in terms of the Dirac measure: for any Borel set $A \subseteq \mathbb{R}^2$, let $\delta_0(A) = 1$ if $0 \in A$, and $\delta_0(A) = 0$ otherwise. In particular, $\int_{\mathbb{R}^2} \delta_0(d\mathbf{x}) = 1$. The infinity/zero expression is only a heuristic, not an ordinary function.

From this, it follows that, for any $\mathbf{x_0} \in \mathbb{R}^2$, we have the so-called *sifting property*:

$$\int_{\mathbb{R}^2} f(\mathbf{x})\delta(\mathbf{x} - \mathbf{x_0})\, d\mathbf{x} = f(\mathbf{x_0}).$$

With this in mind, an $M \times N$ digital (grayscale) image can be represented as an impulse train

$$f_d(x, y) = \sum_{i=1}^{M} \sum_{j=1}^{N} a_{i,j}\delta(x - x_i, y - y_j), \tag{1}$$

where $a_{i,j} = f(x_i, y_j)$, and the (x_i, y_j)s are the sampled pixel coordinates in the original space, $\mathbb{R}^2$. For simplicity, we will assume that $(x_i, y_j) = (i, j)$.

We will write $f[m, n] = a_{m,n}$ for the pixel values, i.e., the weights of the impulses in f_d. Note that m, n may be *any* integers—we set $f[m, n] = 0$ outside the image grid $([1, M] \times [1, N]) \cap \mathbb{Z}^2$. In light of this, we may view $f[m, n]$ as a signal $\mathbb{Z}^2 \to \mathbb{C}$.

Discrete convolution

Perhaps the most central operation in digital image processing is that of convolution. The convolution between two signals $k, f\colon \mathbb{R}^2 \to \mathbb{C}$ is defined as

$$(k * f)(\mathbf{x}) = \int_{\mathbb{R}^2} k(\mathbf{y}) f(\mathbf{x} - \mathbf{y})\, d\mathbf{y}. \tag{2}$$

(As a technical condition, we assume the continuous-domain functions k and f are square-integrable so that the integral above exists.)

We will think of one signal, say f, as an image, and the other signal, k, as a *convolution filter* or *kernel*. Assuming the filter is zero outside $[-K, K]\times[-L, L]$, we define the *discrete convolution* of the coefficient arrays $k[i, j]$ and $f[m, n]$ by

$$(k * f)[m, n] = \sum_{i=-K}^{K} \sum_{j=-L}^{L} k[i, j] f[m - i, n - j]. \tag{3}$$

Here, $(k * f)[m, n]$ denotes the convolution of the discrete arrays, while $(k * f)(\mathbf{x})$ in (2) denotes continuous convolution.

Task I: Image class

We will use the public domain library `stb` to read and write images from and to disk. The relevant header files are provided on Canvas, or you may download them directly from the repository `https://github.com/nothings/stb`.[3]

The `stb` library is header only but contains functions that must be compiled. Because of the one-definition-rule, this requires some care. In this particular case, the library implementers have chosen to add special `#define`s that will turn on the implementation parts (function definitions) during the compilation. This should be done in *one translation unit only*!

Here's an example for how to include the library in one of your `.cpp` files:

[3] We will use only `stb_image.h` and `stb_image_write.h`.

```
#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"
#define STB_IMAGE_WRITE_IMPLEMENTATION
#include "stb_image_write.h"
```

Even though it is possible to include the headers in other files (without the `#define`s), we recommend keeping all core image loading/saving code in a single `.cpp` (of course, you may want to expose functions/classes in your own header file(s) that rely on this functionality, but you do not necessarily need to expose the `stb` library).

Read the documentation in `stb_image.h` (you can stop reading when the UNICODE section starts). In particular, you'll want to understand how the `stbi_info`, `stbi_load` and `stbi_image_free` functions work.

The number of "components per pixel" is the number of channels in the image, e.g., red/green/blue. The loaded image will be in *interleaved* array format, meaning that to access the `k`th channel's color value at pixel `i,j` (where `i` and `j` are the indices in the x and y directions, respectively), you'll need to access the `(j*m_x*m_k+i*m_k+k)`th index in the array, if the image has size `(m_x,m_y,m_k)`, where `m_x,m_y` are the width and the height of the image, and `m_k` is the number of channels.

Each channel value ranges from $0-255$ and occupies one byte, stored using `unsigned char`, but we will want to work with values in the interval $[0,1]$. This means that you should represent the image as an array of `double` or `float`. When reading/writing data, you will therefore need to convert between the appropriate ranges and data types. Kernel entries and intermediate values may lie anywhere in $\mathbb{R}$.

Moreover, we can store each pixel's RGB values in a `struct`, so that it is not necessary to keep track of channel offsets and similar while implementing algorithms, etc..

Some skeleton code for an image and pixel classes is provided on Canvas.

- Implement the functionality to load an image as an instance of the `Image` class, using `stb_image`.

Read the documentation in `stb_image_write.h`. We recommend using the `stbi_write_jpg` function to write an image in JPEG format.

- Implement the functionality to save an image represented by the `Image` class using `stb_image_write`.

Hint: you will have to allocate memory to store the image in `unsigned char` format before saving the image.

- If one has the following code:

```
Image img = loadImageFromFile("file.jpg");
```

  - Does `img` live on the stack or heap?
  - What about the pixel data contained in the image?
  - Explain the ownership semantics (who owns what, is the ownership exclusive or shared) of the image and its pixel data.

Note! The `stb` library does not provide any RAII facilities. You should be careful to clean up after yourself as you work with the library.

Task II: Gamma correction

- Create a function that applies the following transformation on every pixel (same for all channels) of an image:

$$x \mapsto x^{\gamma}$$

  where $\gamma > 0$.

- Try loading an image of your choice, applying the transformation above, and then saving the output on disk. You can use different values of γ, e.g., $\gamma = 0.5, 10$ to see how it affects the image.

Task III: Image transformation class

We will need at least two image transformations for the following to make sense. For this reason, start by implementing a grayscale transformation in a similar way as your gamma correction from above. To accomplish this, the transformation should now be

$$x \mapsto 0.2126x_R + 0.7152x_G + 0.0722x_B,$$

where x_R, x_G, x_B are the red, green and blue components of the pixel x. Use the same mapping for all channels.

We will now create a polymorphic `ImageTransformation` class that can serve as a base for both the gamma correction and grayscale transformations.

- Create an `ImageTransformation` class with a virtual function `apply` that takes an image by non-const reference. Applying an `ImageTransformation` to an image should modify the image that gets passed as the argument.
- Should the `apply` function be pure virtual?
- Create two derived classes `GammaCorrector` and `Grayscaler` whose `apply` methods perform the two image transformations we have developed earlier in this assignment.
- How should you deal with different γ values being passed by the user?
- Introduce a `std::vector` that can store different types of image transformations. Put both the gamma correction and grayscale transformations into the vector. Loop over the vector elements applying each transformation in turn, starting with the input image, and save the resulting image to disk.
- What are the relationships of `GammaCorrector`/`Grayscaler` to their common base class, `ImageTransformation`? (Not simply "inheritance.")

Task IV: Convolution filters

- Create a `SpatialConvolution` class that derives from `ImageTransformation`. This class should hold onto a discrete convolution filter (you can store this as an image, using the same image class as you have used for the other images in the assignment).
- The `apply` function should implement the discrete convolution (Equation (3)) between the input image and the filter stored in the `SpatialConvolution` class. *The convolution should be implemented on a channel-by-channel basis.*
- Create a `LaplaceFilter` class that derives from `SpatialConvolution` that initializes the filter in the `SpatialConvolution` to the discrete 3x3 negative Laplace operator:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}.$$

  (For a more correct result, the "4" should be at the center of the filter.) Hint: you will need to care about what happens at the boundary of the image. Several approaches are available. You can, for example, add zero padding (artificial pixels with all channels set to zero—i.e., black pixels), or mirror the image across its boundary (for example, across a boundary at $x = 0$, set $f(x, y) = f(-x, y)$ for $x < 0$). If you implement the Gaussian blur in Task V, the latter approach may give a better result.
- How does applying `LaplaceFilter` to an image affect it?
- Bonus/optional: a sharpening filter can be obtained by changing the 4 to a 5 in the matrix above. At this point, you should be able to add a `SharpeningFilter` without much effort.

Task V: Gaussian blur (PhD only)[4]

To blur an image, one can convolve it with a centered Gaussian density,

$$k(\mathbf{x}) = \frac{1}{2\pi} \det(\Sigma)^{-1/2} \exp\left(-\frac{1}{2}\mathbf{x}^T \Sigma^{-1} \mathbf{x}\right)$$

where $\Sigma = \sigma^2 I_2 = \begin{bmatrix} \sigma^2 & 0 \\ 0 & \sigma^2 \end{bmatrix}$, where $\sigma^2 > 0$ is a parameter.

Since k has support on all of $\mathbb{R}^2$, we will need to make a cut-off at some point since we do not have infinite memory or time for the computation. For instance, one can set k to zero outside the box $[-3\sigma, 3\sigma]^2$ as the influence of the filter three standard deviations outside the center is so small that leaving anything beyond this will not make a significant difference to the result.

[4] Master's students are of course welcome to complete this part, as well, but it is not required to complete the assignment!

- Assuming an $N \times N$ image, how does the computation time of the discrete convolution scale as σ increases?

A filter is *separable* if we can write it as a product of two simpler filters.

- Show that k can be written as $k_V k_H$ for two one-dimensional filters k_V and k_H represented as a column vector and a row vector, respectively.
- Show that $k * f = k_V * (k_H * f)$.
- What is the time complexity of performing the convolution in two steps: $k_V * (k_H * f)$?

Finally, you will implement a Gaussian blur filter in your program.

- Implement the Gaussian blur first using the convolution operation you have developed in Task IV.
- Next, implement Gaussian blur as a separable convolution.
- Is the second approach faster in practice?

(You may assume exact arithmetic in your proofs.)

References

[1] Rafael C Gonzalez and Richard E Woods. *Digital image processing*. Pearson, 4th edition, 2017.

Submission

The programming exercises should be done individually, or in groups of two. Hand in a report containing:

- Comments and explanations that you think are necessary for understanding your program. (But do not comment excessively.)
- The output of your program according to the tasks. Don't forget to draw conclusions!
- Source code included as an appendix to your report. (We need this to comment on your code. Points will be deducted if the source code is missing from your report.)

In addition, all source code for your program(s) in .zip/.tar format and instructions for how to compile/run.

Important! Your submission must contain at least the following two files:

- PDF report (not zipped!) with your findings and source code
- .zip/.tar archive containing your source code

Submissions not following this format will not be accepted and will have to be resubmitted (these will be considered late submissions).

Good luck!